

CSCE 670 INFORMATION RETRIEVAL · MODULE 04 · LECTURE NOTES

Query Processing

Making good ranking affordable.

MODULE OUTCOMES

- Explain term-at-a-time and document-at-a-time processing.
- Explain the safe early-termination strategies.
- Explain the unsafe ones and what they cost.
- Explain caching and index partitioning.
- Measure and meet a latency budget.

1 Two orders of processing

TERM-AT-A-TIME

process one query term's entire postings list, accumulating partial scores in a hash table of candidate documents, then move to the next term. Memory: one accumulator per candidate document, which can be a large fraction of the collection.

DOCUMENT-AT-A-TIME

advance all the query terms' postings lists together in document order, completing each document's score before moving on. Memory: one heap of size k . Which is why it won.

AND DOCUMENT-AT-A-TIME ENABLES EARLY TERMINATION

because a document's score is final when you leave it, so you can compare against the current k -th best and stop when no remaining document could possibly beat it (section 2).

TERM-AT-A-TIME CANNOT DO THIS, since no document's score is final until every query term has been processed.

Document-at-a-time's bounded memory and its support for early termination are the same property — scores complete as you go, which is what both lets you discard a document after scoring it and lets you reason about whether remaining documents could qualify. **Connecting the two advantages to one cause is the insight**, and it explains why the choice was not a close one once collections grew.

2 Safe early termination

WAND SKIPS DOCUMENTS THAT PROVABLY CANNOT ENTER THE TOP K

Precompute, for each term, the maximum score it could contribute to any document — its upper bound, which depends only on the term's IDF and the largest term frequency anywhere in its postings list, and is computed once at index time.

Then at each step, sum the upper bounds of the terms whose postings lists are currently positioned at or before the candidate document. If that sum is below the current k-th best score, then no document at that position can possibly qualify — so skip the lists forward past it.

Which makes it *safe*: the resulting top k is identical to what exhaustive scoring would produce, because every document skipped was provably ineligible rather than merely unpromising.

And block-max variants tighten the bounds considerably by storing a maximum score per postings block rather than one per term — a much tighter bound, and therefore much more skipping, which is what current implementations use.

- **A safe optimisation returns exactly the same top k, so it can be deployed without re-evaluating the system's quality at all** — which is its organisational value as much as its technical one.
- **And it can be verified mechanically: run both the optimised and the exhaustive version and compare the result lists** — which is **Project 1's requirement** and takes minutes to set up.
- **An unsafe optimisation changes the results**, so it requires a fresh evaluation (Module 05) — and **a difference in the metric could be the optimisation's effect or could be noise**, which makes the assessment much harder.
- **Which makes safety a property worth paying for**, because **it decouples performance work from quality work** — the team can optimise freely without re-running the evaluation, which changes how quickly they can iterate.
- **And an unverified 'safe' optimisation is a bug waiting** — the bound arithmetic is easy to get subtly wrong, **and the comparison against exhaustive scoring is the only thing that establishes correctness**, which is why Project 1 grades it.

3 Unsafe optimisations

- **Static index pruning** — remove postings that could rarely contribute to a top-k result, which **shrinks the index permanently** and **may lose a rare but entirely correct answer** for an unusual query.
- **Tiering** — search a small, high-quality tier first and fall back to the full index only if too few results are found, **which works well in practice** and **systematically disadvantages the lower tier's documents** in a way that compounds over time (Module 12 §2's exposure argument).
- **Champion lists** — keep only each term's highest-scoring documents in a separate structure, and score only those for most queries.
- **A document-count cutoff** — simply stop after examining n documents, **which bounds the latency directly** and is **sensitive to the order documents appear in the index**, which makes the quality effect depend on something arbitrary.
- **And all of them are legitimate techniques** — **provided the quality cost is measured rather than assumed negligible. Unsafe does not mean wrong**: it means **the quality effect has to be measured**, which is a requirement rather than an objection, and the measurement is Module 05's apparatus.

4 Systems

QUERY DISTRIBUTIONS ARE HEAVY-TAILED, WHICH MAKES CACHING EXTREMELY EFFECTIVE

A small number of distinct queries account for a large fraction of all traffic — the same heavy tail as everywhere else in this semester — so a result cache of quite modest size serves a substantial share of requests at almost no cost.

And there are two caches worth having: a result cache keyed on the whole query, and a postings cache for frequently-accessed lists — the second of which helps the long tail of unique queries, since even unique queries share common terms.

With the invalidation question being the genuinely hard part: a cached result is stale the moment the index changes, and the acceptable staleness depends entirely on the application — a news search and a documentation search have very different answers.

Which is the same heavy-tail-plus-caching pattern as everywhere else in systems — and here the tail is in the query distribution rather than in the data, which is worth noticing because it means the caching opportunity exists even when the collection is uniform.

- **Document partitioning**: each machine holds a subset of the documents and runs the complete query against its own shard, with the results merged centrally. Simple, naturally balanced, and the standard choice.
- **Term partitioning**: each machine holds the complete postings for a subset of terms, so a query only touches the machines holding its terms — **less total work, and badly imbalanced** by the heavy-tailed term distribution, with the common terms' machines overwhelmed.

- **So document partitioning wins in practice**, because **balance matters more than total work when the latency is set by the slowest machine** — which is the whole argument.
- **Which is CSCE 676 Module 10 §2's skew argument**, arriving here with a concrete and consequential design conclusion rather than as a general warning.
- **And it makes the tail latency the thing to measure**, since **the query cannot return until every partition has responded**. With n partitions, the query latency is the maximum of n samples from the per-partition latency distribution — so **the more you partition, the worse the tail gets**, which is the fundamental tension in this design and is why very large fan-outs need hedged requests.

AND THE LATENCY BUDGET IS A DESIGN INPUT

Decide the budget first, because **it determines the architecture** — **how many candidates you can afford to retrieve**, **how expensive a reranker you can run over them** (Module 06 §4), and **whether a neural model is usable at all** at your query volume.

Measure the distribution rather than the mean. The 95th and 99th percentiles are what users experience as the system's speed, and **the mean hides them entirely** — particularly with the partitioning fan-out above.

And budget per stage: retrieval, reranking, and presentation each receive an explicit share — which makes the trade-offs visible and negotiable rather than discovered when something is too slow.

Which is the module's practical conclusion: a ranking function you cannot evaluate within the budget is not a ranking function you have — so **the budget constrains the quality work rather than following it**, and that ordering is what Module 06 §4's two-stage architecture exists to exploit.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Manning, Raghavan & Schütze, chapter 7 (free)	§1 through §3 — the processing orders and the inexact top-k strategies.
Broder et al. — Efficient query evaluation using a two-level retrieval process (free)	§2's WAND in the original — short, and the bound argument is the whole paper.
Ding & Suel — Faster top-k document retrieval using block-max indexes (free)	§2's tightening — block-max WAND, which is what current engines implement.
Dean & Barroso — The Tail at Scale (free)	§4's fan-out problem — why partitioning worsens the tail, and what to do about it.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement both processing orders** and compare their peak memory.
- 2 **Explain why term-at-a-time cannot terminate early.**
- 3 **Compute per-term upper bounds** at index time.
- 4 **Implement WAND** and measure the fraction of documents skipped.
- 5 **Verify the top k matches exhaustive scoring exactly**, on a thousand queries.
- 6 **Add block-max bounds** and remeasure the skipping.
- 7 **Implement a document-count cutoff** and measure both the latency gain and the metric loss.
- 8 **Measure your query distribution's skew** and compute the hit rate of a cache holding the top 10,000 queries.
- 9 **Partition an index by document and by term**, and compare the load balance.
- 10 **Report your latency at the median, 95th, and 99th percentiles.**

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Contrast the two processing orders on memory, and say why one won.
- 2 Why does document-at-a-time enable early termination?
- 3 Explain WAND's bound and why it is safe.
- 4 What do block-max variants improve?
- 5 Why does safety decouple performance from quality work?
- 6 How do you verify a safe optimisation?
- 7 Name four unsafe optimisations and what each costs.
- 8 Why is caching so effective here, and what are the two caches?
- 9 Contrast document and term partitioning, and say which wins and why.
- 10 Why does partitioning worsen the latency tail?